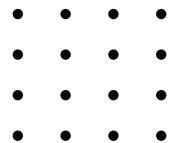
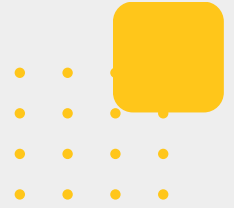
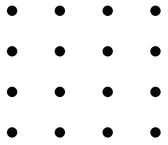


Final PPT

IoT Testbed

Group G-08





Tech Stack



Backend: SQLite, Postgres(future), Redis, FastApi, Python



Frontend: Node.js, React.js, JavaScript, Vite



Data Models



User



- Registration details
- User Credentials

Gateway

Raspberry Pi Devices



- Name
- IP address
- Token (hashed)
- Verification status
- Last seen timestamp

Device

IoT device
connected to
gateway

- Status
- Last seen timestamp





Data Models



Job Group

- Collection of user-submitted jobs
- Tracks status & timestamps

Job

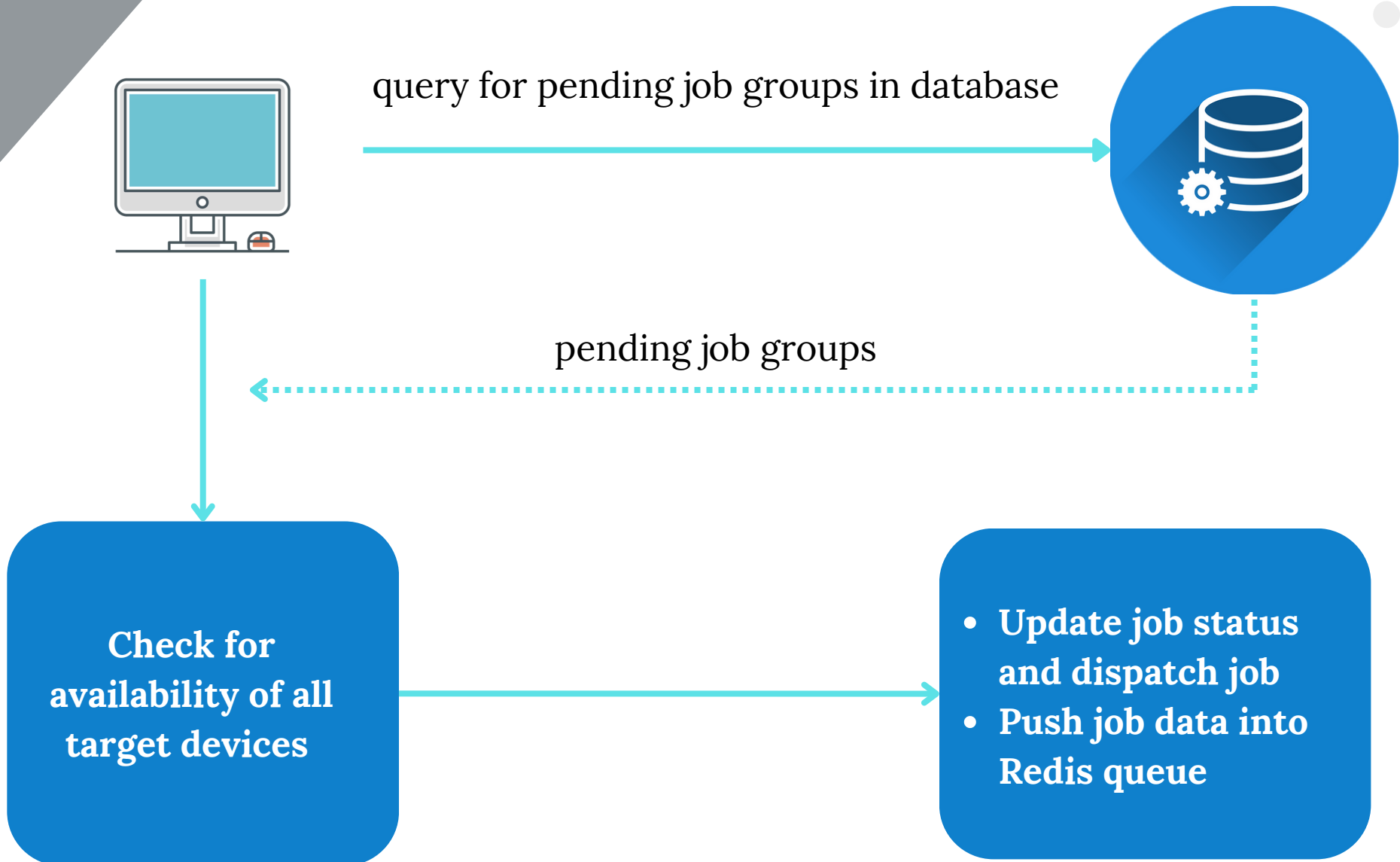
- Job records reference an executable (source_file_id) and optional output (output_file_id)
- Tracks job status and execution timestamps.

File

- Filename
- Filepath
- Upload timestamp



Scheduler



Scheduler

Check Database
for Pending Job Groups



Dispatch Jobs
Target Devices/Gateways Availability



Validate
Update Statuses Push Job Data into Redis



Job Execution
Devices/Gateways Process Jobs



Priority & Estimated Runtime

- Specify job priority and estimated runtime
- Order or preempt jobs based on available time slots.

Advanced Scheduling

- free intervals; use optimization
- Event-driven triggers for scheduling

Potential Enhancements to Scheduler



Priority & Estimated Runtime

- Users specify estimated runtime and job priority.
- Scheduler orders or preempts jobs based on priority and time slots.



Advanced Scheduling Algorithms

- Store device busy periods and planned job executions.
- Use historical data for optimal pre-calculated scheduling.



Pre Scheduling

- Track busy/free time intervals for each device.
- Optimize job allocation to minimize downtime and improve resource use.



Event Driven

- Switch from periodic polling to an event-driven model.
- Trigger scheduling decisions on state changes

API Summary



Tech Stack:

- FastAPI with Pydantic (for request validation/serialization)
- JWT-based Authentication (for secure access)

Authentication:

/auth/register,
/auth/login,
/auth/me

File Management:

/files/upload,
/files/{id}/download,
/files

Gateway Management:

/gateways/ (create new gateways),
/gateways/register,
/gateways/heartbeat

Device & Job Management:

/devices,
/job-groups,
/jobs

Future Enhancements:

- Expand endpoints for advanced job monitoring and device management.
- Integrate role-based access control for admin operations.
- Enhance performance and security as the system scales.

Message Queue

- **Job Dispatch:**

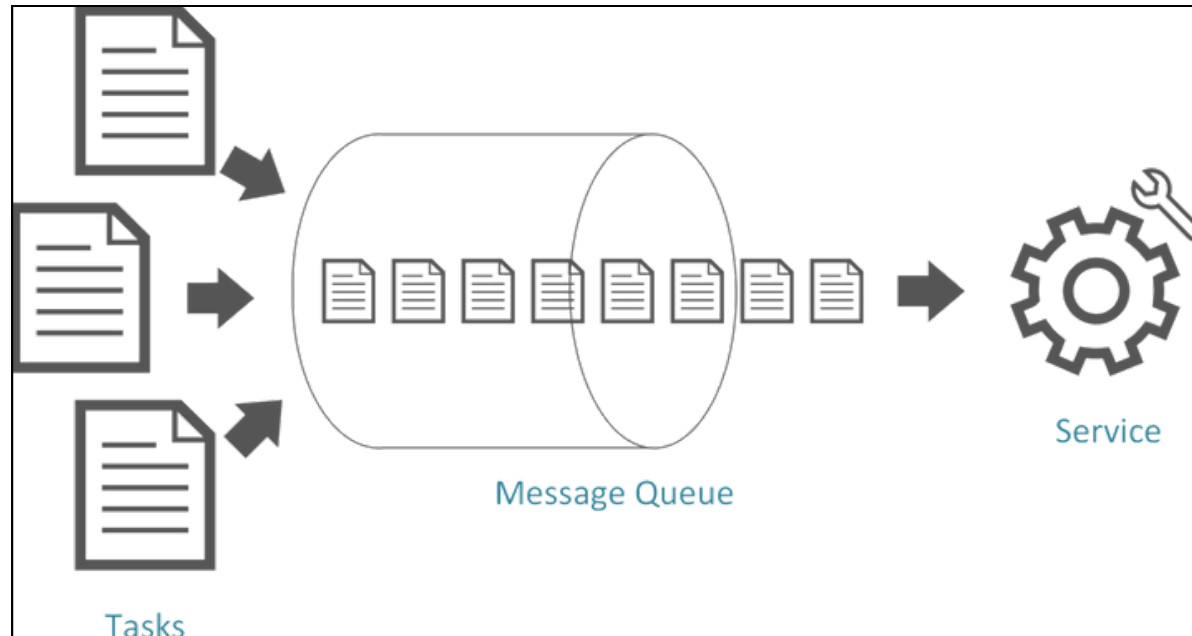
Scheduler pushes jobs into a Redis list (keyed by gateway ID).

- **Asynchronous Retrieval:**

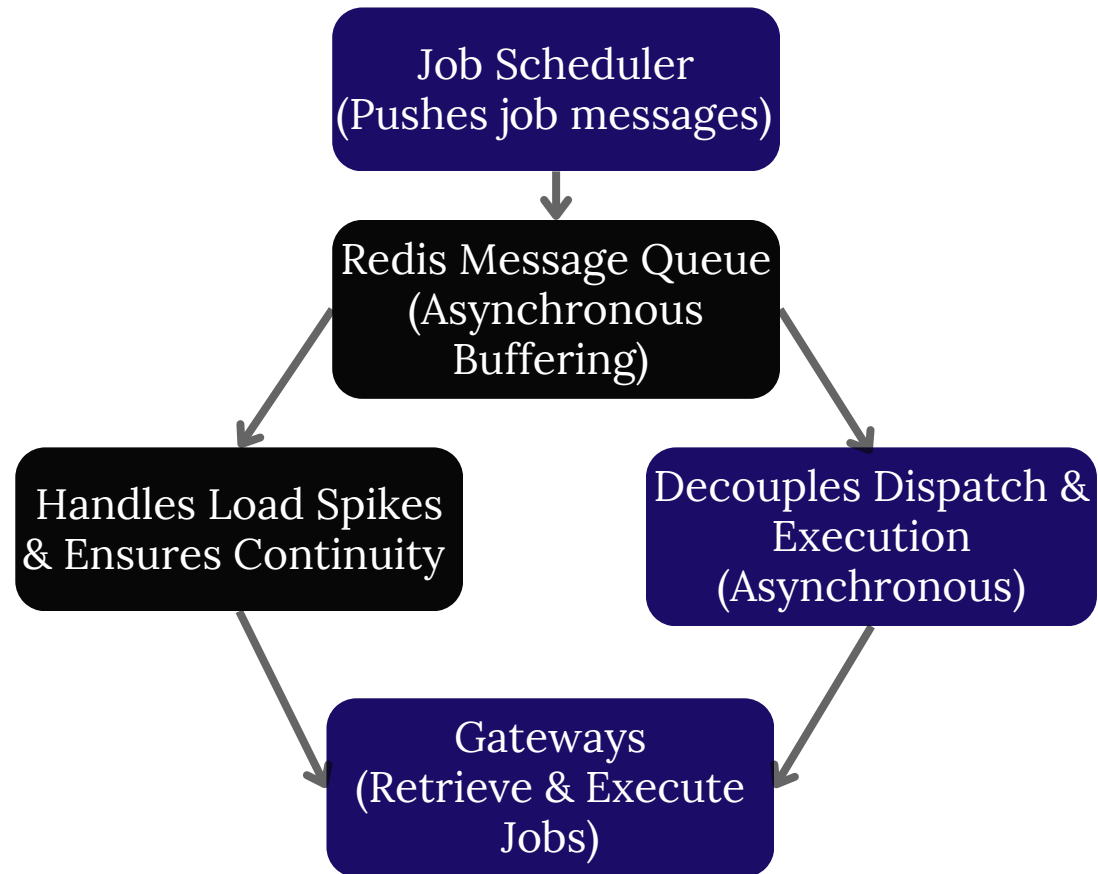
Gateways use blocking pop to fetch jobs.

- **Decoupling & Buffering:**

Redis buffers jobs to handle load spikes.



Message Queue



Future Enhancements

- Job Prioritization:
 - Urgent jobs get higher priority.
- Message Reliability:
 - Implement TTL to remove stale messages.

Gateways and Gateway Client



Registration & Verification:

- Gateways are added via a POST `/gateways/` endpoint by providing a name (and optionally an IP).
- The backend generates and returns a registration token (with its hash stored) for the gateway.
- The gateway client then verifies itself using the token and name via POST `/gateways/register` to activate its status.

Heartbeat Mechanism:

- Once active, gateways periodically send heartbeat messages via HTTP to update their last seen timestamp and device status.

Job Dispatch & Communication:

- Gateways listen to a Redis-based message queue to retrieve and execute jobs.
- After execution, they report their status back via HTTP endpoints.

Future Enhancements:

- Local Gateway Management: Enhance clients for auto-detection of local devices and support containerized job execution.
- Improved Security: Implement measures such as mutual TLS or certificate-based authentication to secure registration and heartbeat processes.

Gateways & Gateway Client Workflow

Future Enhancements

- Local Gateway Management (Auto-detect & container support)
- Improved Security (mTLS, certificate-based auth)

01

Gateway Registration & Token

- API: POST /gateways/
- Input: Name ($\pm$ IP)
- Output: Registration Token

02

Gateway Verification

- API: POST /gateways/register
- Uses: Token & Registered Name
- Outcome: Status updated to Active

03

Heartbeat Mechanism

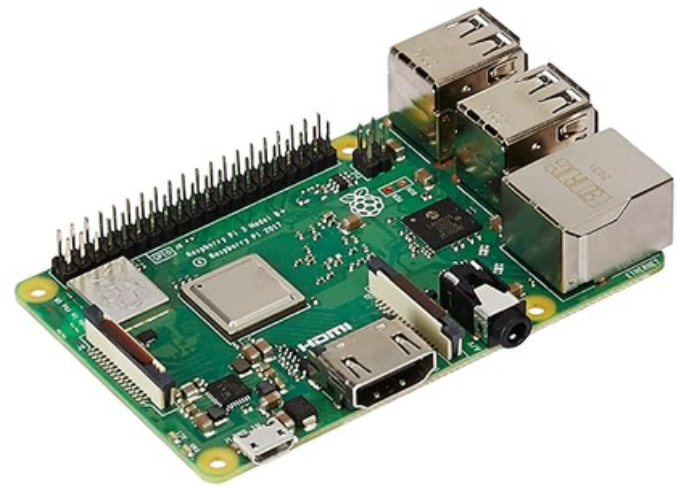
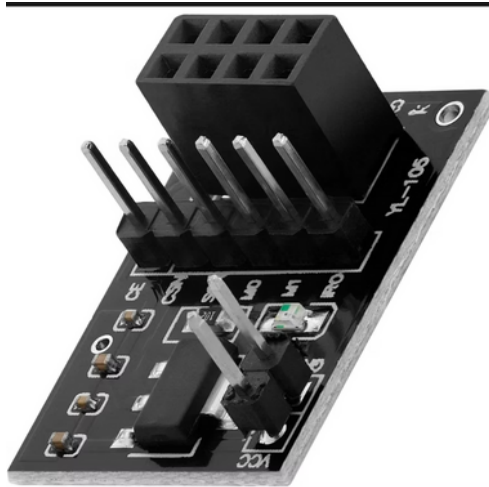
- Periodic HTTP heartbeat calls
- Updates: Last seen & device info

04

Job Dispatch & Communication

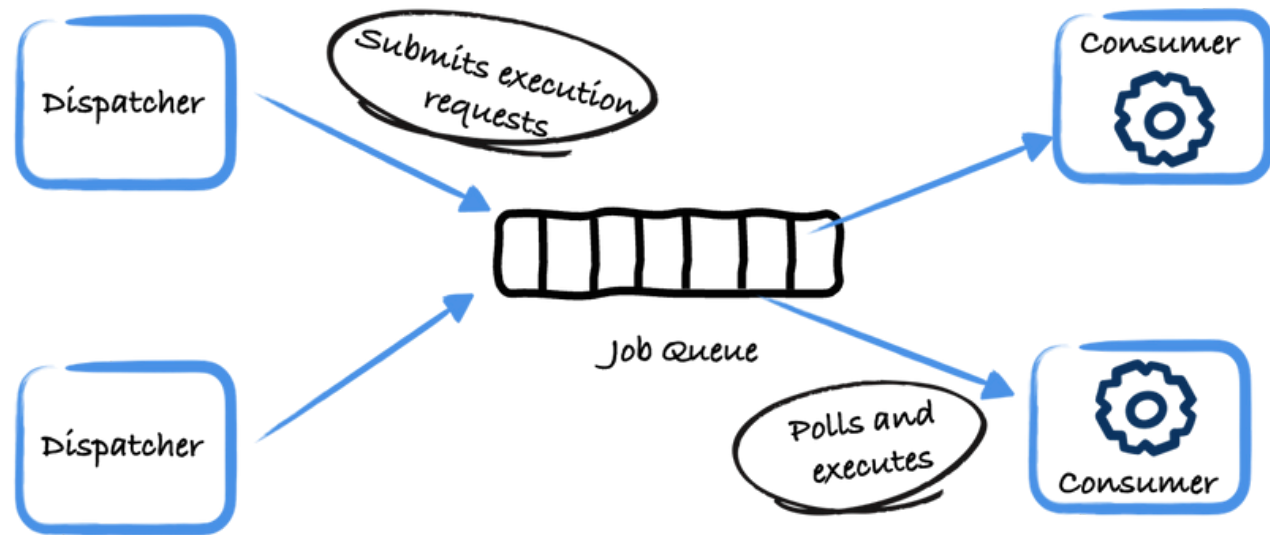
- Listens to Redis message queue
- Retrieves, executes, and reports job statuses via HTTP

Device Registration to Gateway



On the gateway side whenever a device needs to be registered to a gateway, the gateway itself identifies the devices connected to it, and user can easily register it by selecting the index and entering the device name.

Gateway Job Download



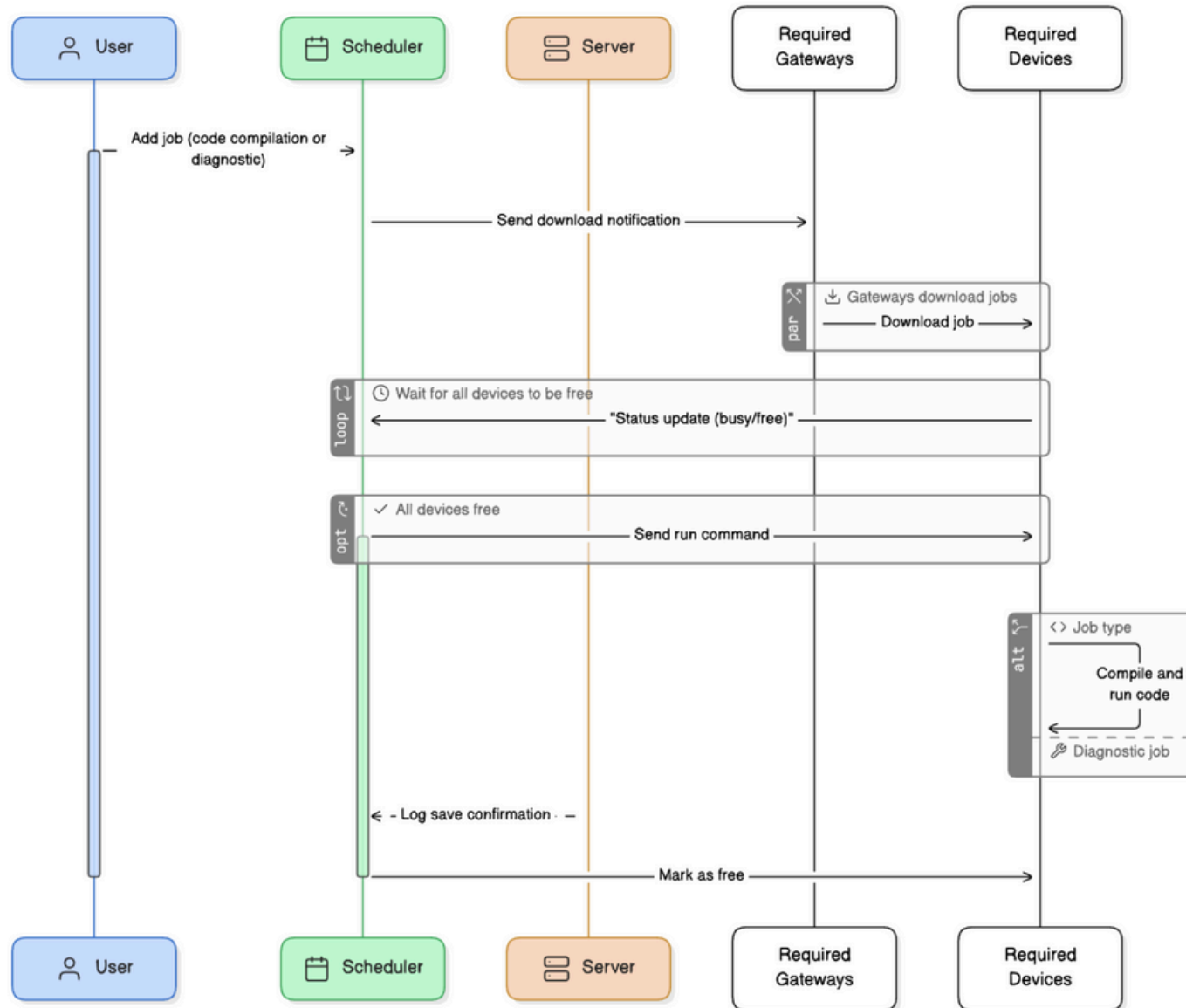
- Gateway side continuously runs a thread that polls the notification buffer for the jobs.
- As soon a job notification is present for it , it downloads the job from the server



Job Execution - Device

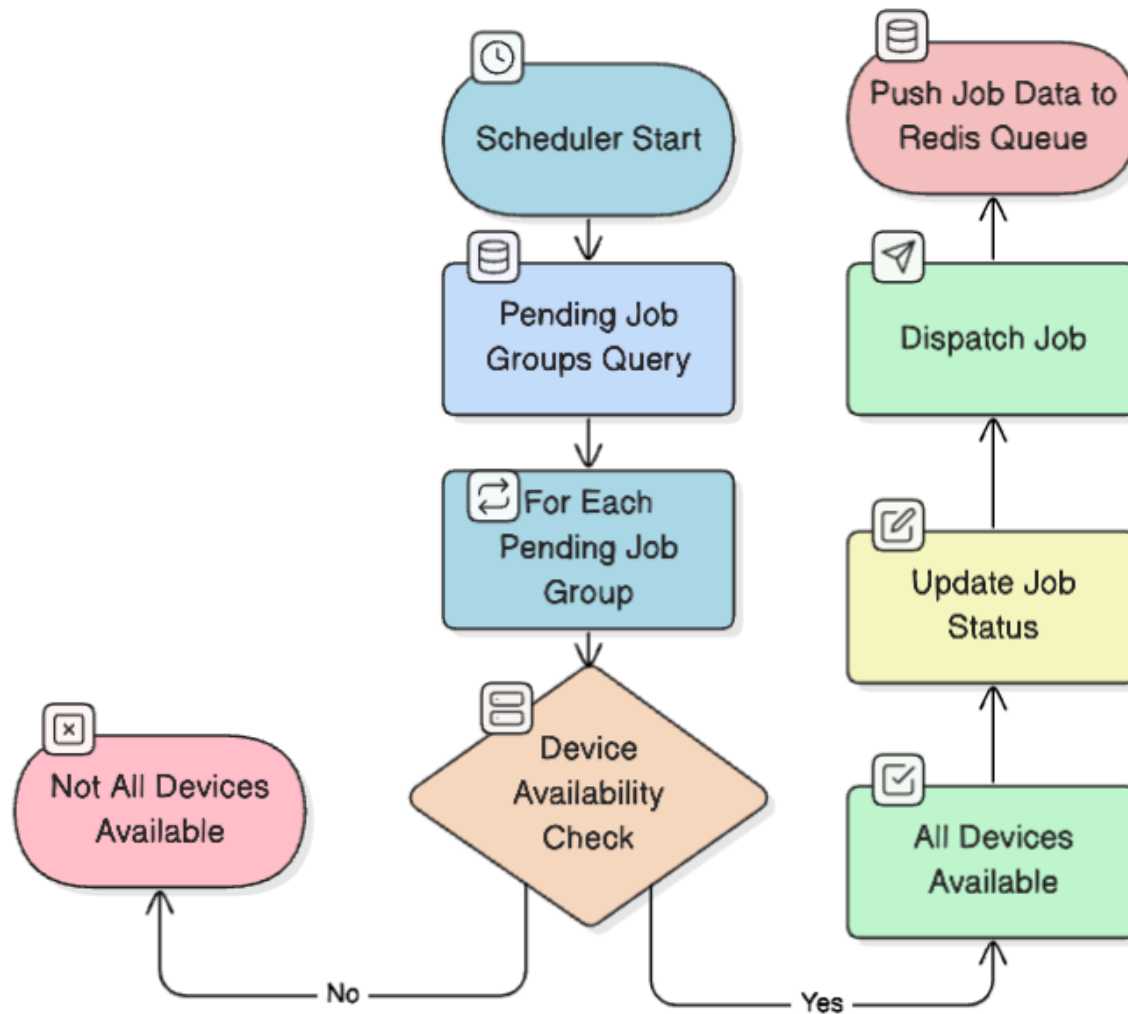


Job Submission and Execution Flow



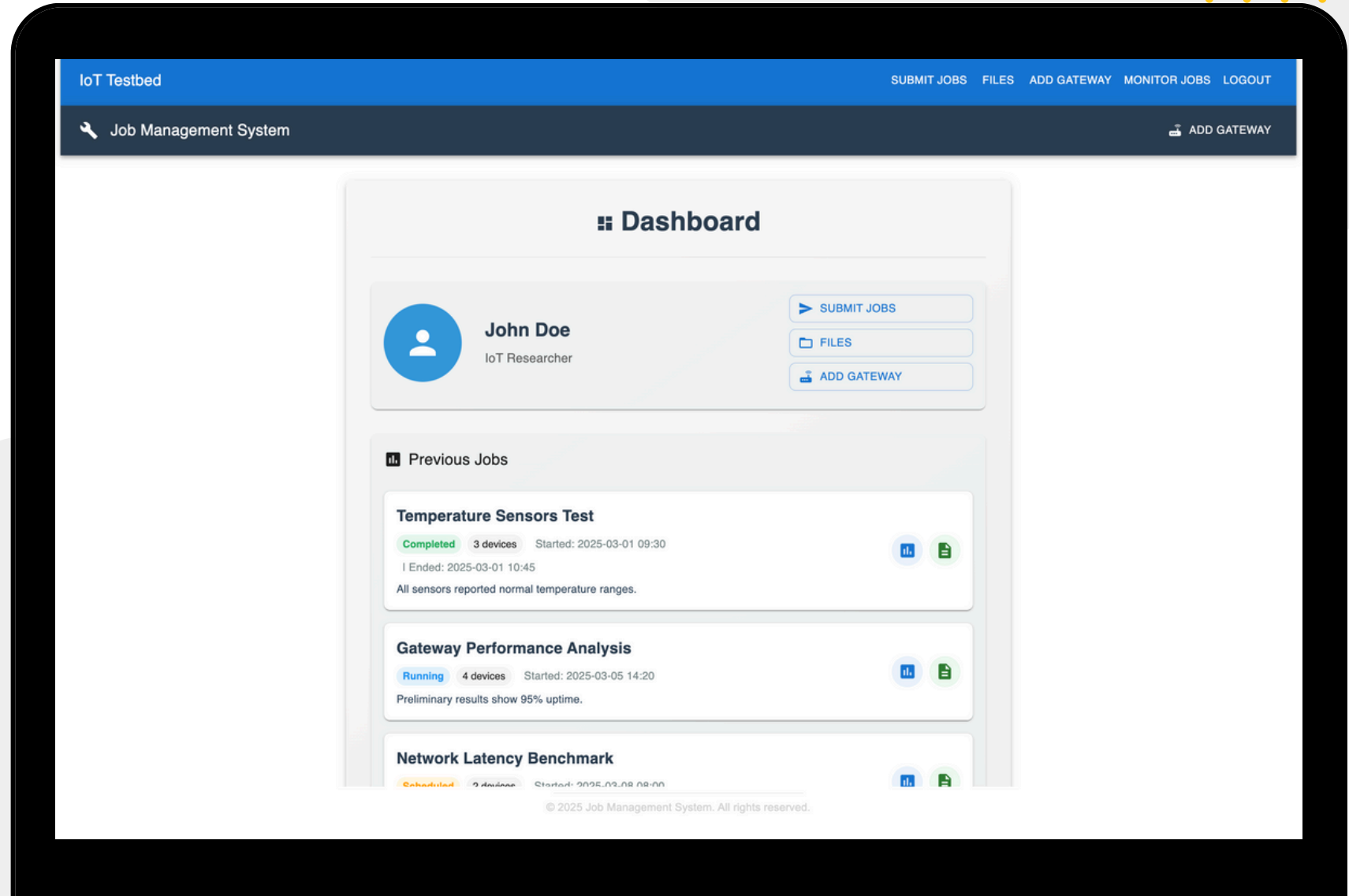
Job Execution - Scheduler

Workflow for Processing Pending Job Groups



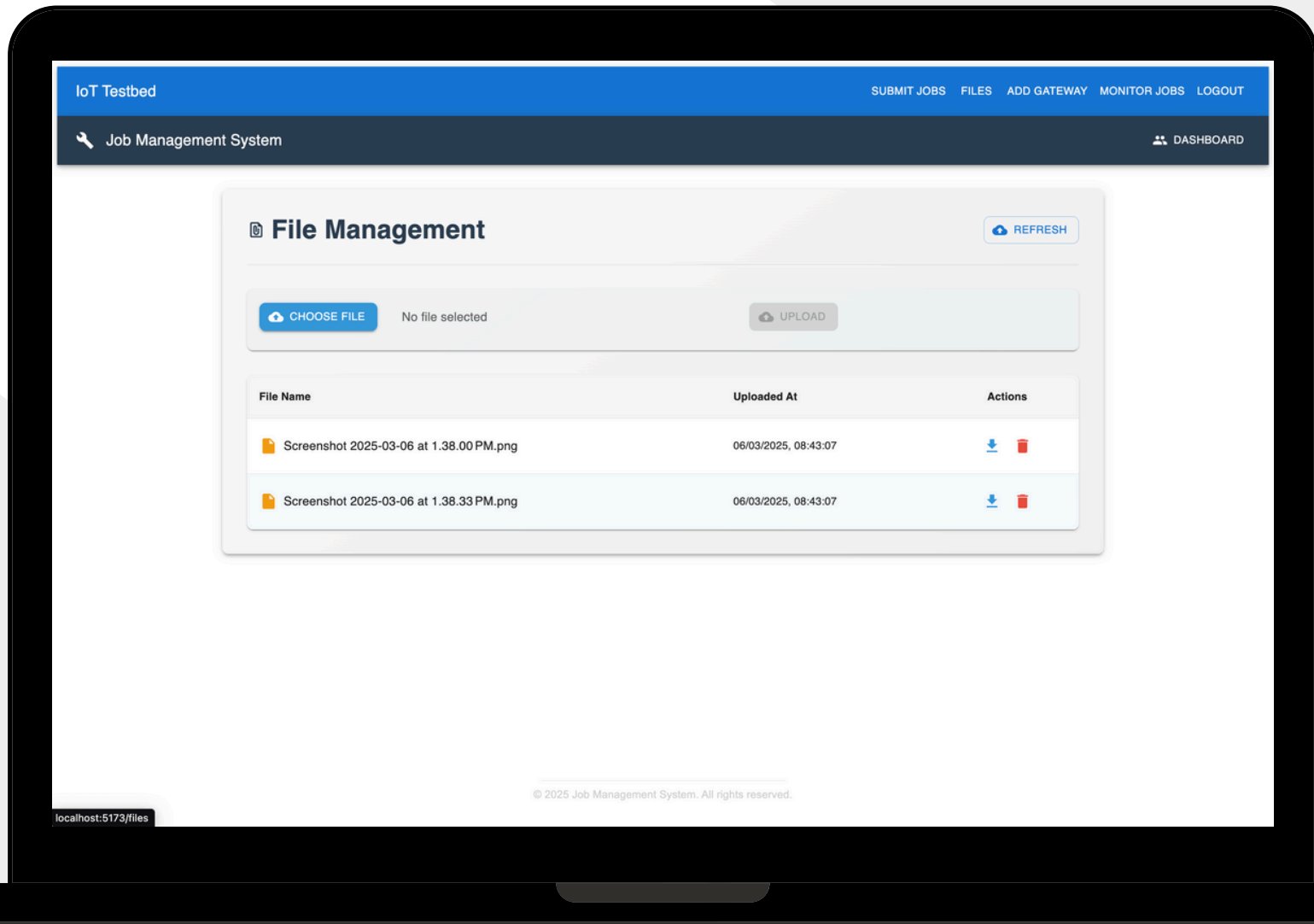
Front End Pges

DashBoard



File Management:

- Enables users to upload, list, download, and delete files.
- Integrates with backend file endpoints to handle file operations.



Job Submission

- Provides a form for submitting job groups by selecting target devices (or gateways) and associating an executable file with each job.
- Dynamically supports adding/removing job entries, ensuring all required fields are populated before submission.

The screenshot displays the 'IoT Testbed' web interface. The top navigation bar includes links for 'SUBMIT JOBS', 'FILES', 'ADD GATEWAY', 'MONITOR JOBS', and 'LOGOUT'. The main header shows 'Job Management System' and a 'DASHBOARD' link. The central form is titled '> Submit New Job Group' and contains the following sections:

- Job Group Name***: A text input field containing 'new-experiment'.
- Available Devices**: A section with four checkboxes labeled 'd1', 'd2', 'd3', and 'd4'. The 'd1' checkbox is checked.
- Assign Files to Selected Devices**: A section showing 'Selected devices: 1' with a list containing 'd1'. Below this is a dropdown menu labeled 'File for d1'.

At the bottom of the form, a copyright notice reads: '© 2025 Job Management System. All rights reserved.'

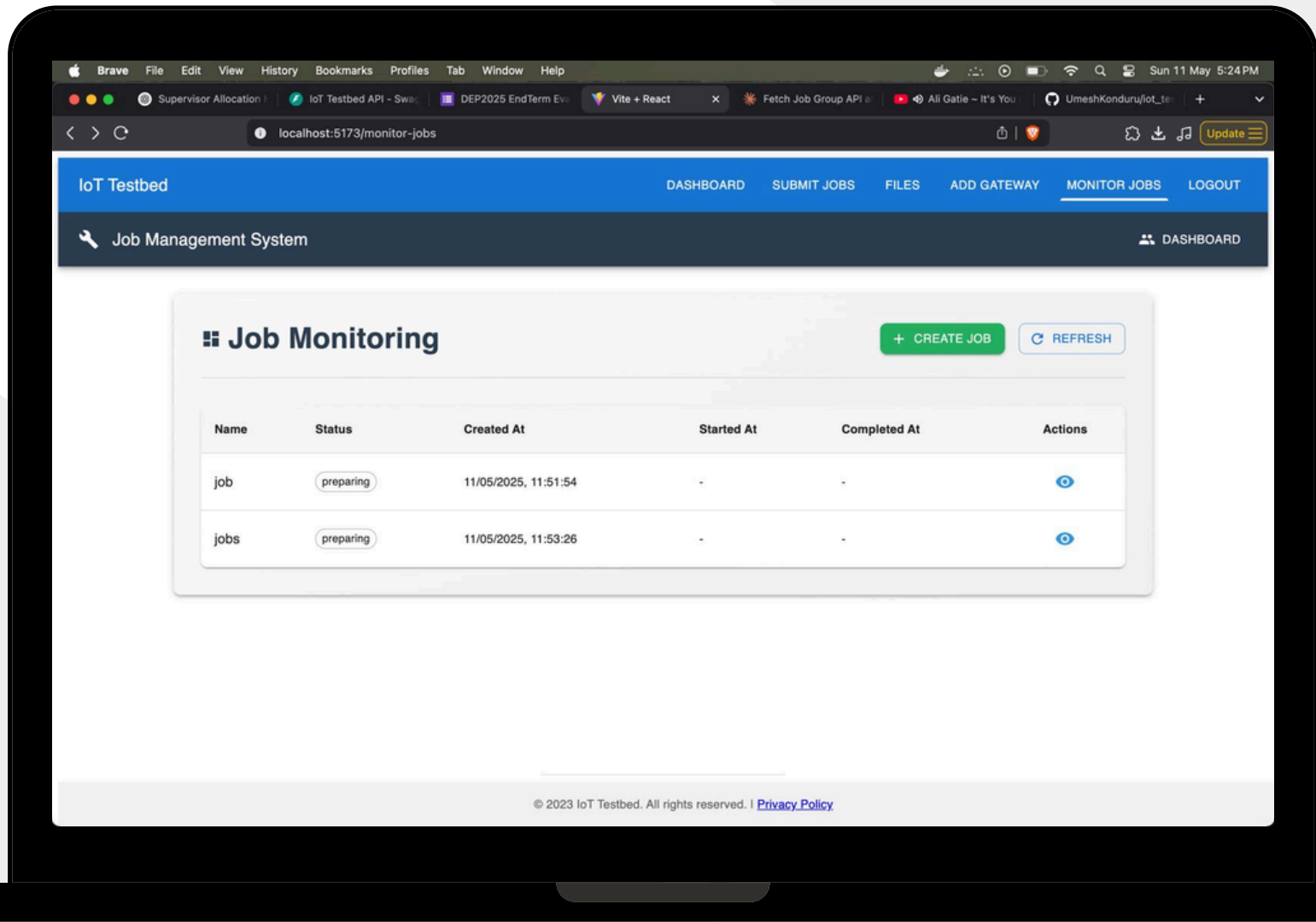
Gateway Management:

- Offers page to add new gateways by entering a gateway name (and optionally IP address).
- Displays the generated registration token (and optionally gateway name/ID) for use in gateway client registration.

The screenshot shows a web application interface for an IoT Testbed. The top navigation bar is blue with the text 'IoT Testbed' on the left and links for 'SUBMIT JOBS', 'FILES', 'ADD GATEWAY', 'MONITOR JOBS', and 'LOGOUT' on the right. Below this is a dark blue header with a wrench icon and the text 'Job Management System' on the left, and a 'DASHBOARD' link with a group of people icon on the right. The main content area is titled 'Add Gateway' with a small gateway icon. It contains a form with a 'Gateway Name *' input field. Below the input field is a green button with a gateway icon and the text 'CREATE GATEWAY'. Further down, there is a section titled 'Registration Token' which displays a token 'gAZYG7uJ60q_GeJKa7XsQ' in a light green box. To the right of the token is a copy icon. Below the token box, it says 'Use this token to register your gateway with the system.' At the bottom of the page, there is a small copyright notice: '© 2025 Job Management System. All rights reserved.'

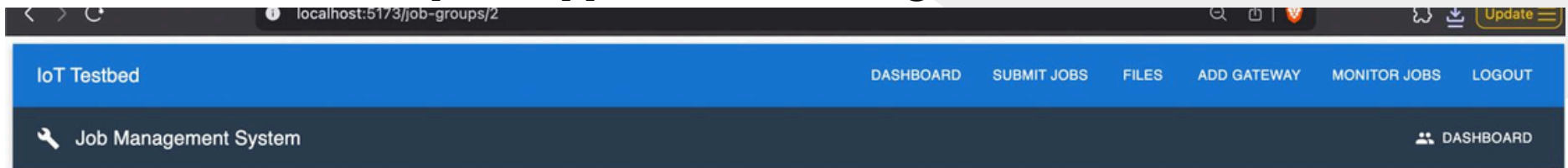
Monitor-Jobs

Page to view job groups and the respective status of individual Job group



Job group Details

- In this page Users will be directed after selecting the required job group , where they can view all the jobs associated to that particular group , and the download option appears once the logs are available.



Dashboard / Job Monitoring / Job Group Details

[← BACK TO LIST](#) **Job Group Details** [REFRESH](#)

jobs

Status: preparing

ID: 2

User ID: 1

Created: 11/05/2025, 11:53:26

Started: -

Completed: -

Jobs (2)

Job ID	Device ID	Status	Created At	Started At	Completed At	Actions
2	1	preparing	11/05/2025, 11:53:26	-	-	↓
3	2	preparing	11/05/2025, 11:53:26	-	-	

Gateway Registration

After getting the token from the server , this page facilitates the registration of gateway to the server.

Gateway Portal

 Dashboard

 Gateways

 Devices

Gateway Registration

Register a new IoT gateway to your network infrastructure

Gateway Name

Registration Token

 Register Gateway

Device Registration

- This page handles the addition of device to the gateway and to notify the server.
- this allows the user to select and name devices form the list of available devices.

Gateway Portal

- Dashboard
- Gateways
- Devices

Device Registration

Connect and register a new IoT device to your gateway

Connected Devices

- ✓ Select a device
- 1. /dev/cu.wlan-debug - n/a
- 2. /dev/cu.debug-console - n/a
- 3. /dev/cu.ZEB-PARADISE - n/a
- 4. /dev/cu.Airdopes311Pro - n/a
- 5. /dev/cu.Bluetooth-Incoming-Port - n/a

Device Name

Register Device

Connected Devices



This shows the list of connected devices to the gateway

Gateway Portal

Dashboard

Gateways

Devices

Connected Devices

Manage and monitor all your IoT devices

Add New Device

Search devices...

All Status

2 devices

Temperature Sensor - Basement

/dev/cu.usbmodem1320

ID: 1

Description: Virtual temperature sensor for testing

Last Seen: 23/04/2025, 12:33:24

Details

Connected

Humidity Sensor - Bedroom

/dev/cu.usbserial-3132

ID: 2

Description: Virtual humidity sensor for testing

Last Seen: 30/03/2025, 12:33:24

Details

Disconnected

Efforts

Stages of completion

- This project is hardware related
- it involved multiple components working in coordination for smooth functioning.

Scheduler & Database

01

- They had to work closely to maintain the records of multiple submitted jobs
- Scheduler also used database to find what devices are free at the moment in order to dispatch the jobs effectively.

Redis Queue

02

- It had the main purpose in effective communication between gateway and server.
- It acts like a temporary buffer of notifications for the gateway.

Server Side

03

- It had to manage various gateways and devices connected to them.
- It had to accept various codes submitted by clients and dispatch them.

Gateway side

04

- It involved connection of device to gateway
- Polling the notification.
- Compilation and Running of Codes from server.

Individual Contribution

This project is a joint effort, and most of the work was done collaboratively as a group. Clear boundaries of individual contributions cannot be distinctly established.

- https://github.com/UmeshKonduru/iot_testbed_dep
- https://github.com/utkarshpatel7356/iot_gateway

Umesh Konduru(2022csb1139)

- Backend of Server side
- Documentation
- Scheduler Design
- Redis Queue setup

Utkarsh Patel(2022csb1140)

- Backend of Gateway side
- Presentation
- Scheduler Design
- Auth Setup

Smriti Gupta(2022csb1128)

- Frontend of Server side
- Presentation
- Database Schema Design of server side

Satvik Srivastav(2022csb1120)

- Frontend of Gateway side
- Documentation
- Database Schema Design of gateway side



Thank You



2022csb1120

2022csb1128

2022csb1139

2022csb1140

